BURO

типизированная память рабочих сущностей

сведения, правила и решения по конкретным рабочим объектам, доступные между сессиями.

бюро предоставляет нейросети единый реестр рабочих сущностей: проектов, сервисов, компьютеров и других конкретных объектов пользователя.

каждая запись хранит сведения о своём объекте: его назначение, расположения, связи, принятые решения, правила и ограничения. эти сведения сохраняются между сессиями и доступны независимо от того, в каком проекте работает агент.

система buro связывает реестр с нейросетью и её харнессом: даёт **CLI (интерфейс командной строки)**, местную или общую память и проверяемый способ её изменения. через CLI агент или юзер управляет бюро командами в терминале.

записи ведёт агент или юзер. всё устроено так, чтобы юзер мог заниматься своими делами и работать, в то время как нейросеть выполняет работу по сохранению и обновлению полезной памяти.

оглавление

PREPROCESSING

подключение

INTRAPROCESSING

команды и записи

POSTPROCESSING

обновление памяти

оглавление

Нажмите на название главы или используйте закладки PDF.

01	что именно хранит бюро	3
02	установить и подключить	4
03	подключить нейросеть	5
04	контекст в начале сессии	6
05	команды и записи	7
06	обновление памяти	8
07	коробочные типы	9
08	проект и сервис	10
09	поля коробочных записей	11
10	поля устройства	12
11	имена и постоянные ID	13
12	связи между записями	14
13	создать свой тип	15
14	как описать поля	16
15	свой тип: книга	17
16	перенести запись в другой тип	18
17	сохранность данных	19
18	копии и восстановление	20
19	местная и общая память	21
20	обновления и подробности	22

что именно хранит бюро

бюро хранит сведения о конкретных сущностях. одна запись описывает один объект: сайт, книгу, сервис, компьютер или что-то другое, с чем работает юзер. агент открывает запись, когда сведения об этом объекте нужны для задачи.

Понятие	Описание
`сущность`	Сам объект: например, книга «История города».
`запись`	Сохранённые сведения именно об этой книге: требования к тексту, решения и ссылки на материалы.
`тип записи`	Определение её полей. Например, какие сведения мы храним обо всех книгах и что означает каждое поле.

тип задаёт **формат и смысл полей**. BURO проверяет формат значений; пояснения к полям помогают агенту понять, что туда писать. программа не проверяет правдивость сведений за человека.

можно начать с одного правила

```
id: my-site
name: Мой сайт
kind: project
important:
  - Публиковать только после моего согласия.
```

у коробочных типов обязательны только id, name и kind. агент может подготовить их сам. остальное заполняется по делу; имя и одно правило уже дают полезную память.

в своей области бюро может заменить разрозненные AGENTS.md, проектные файлы памяти и RAG-поиск контекста об отдельных рабочих объектах. инструкция в промпте или skill объясняет агенту, когда обращаться к бюро. само бюро не запускает нейросеть и не обрабатывает чаты в фоне.

установить и подключить

нужен **Node.js 24.14 или новее**. установите пакет из публичного релиза, создайте память и подключите свой харнесс:

```
npm install -g \
  https://github.com/Krablante/buro/releases/download/v3.1.0/buro-3.1.0.tgz
buro init
buro connect opencodez
```

init создаёт память с готовыми типами. для местной базы он также настраивает расписание резервных копий через планировщик устройства. настройки, база и копии лежат вне установленного пакета.

Что	Стандартное расположение
настройки	`~/.config/buro/config.json`
база	`~/.local/share/buro/state/buro/buro.sqlite3`
копии	`~/.local/share/buro/state/buro/backups/sqlite/`

для обычного местного старта менять config.json не требуется. адрес сервера указывают, когда нужна общая память.

после подключения

вместо opencodez можно указать opencode, codex или claude. OpenCode/OpenCodez нужно перезапустить. в локальном Codex проверьте и разрешите установленный hook через /**hooks**, затем начните новую сессию. в Claude Code начните новую сессию; SessionStart должен быть разрешён.

проверьте `buro current` и `buro help`. первая команда открывает стартовые сведения, вторая показывает команды, подключённую память и её действующие определения.

подключить нейросеть

> **агенту нужны инструкции о BURO и доступ к его командам.** сама установленная программа ещё не объясняет нейросети, когда читать и обновлять память.

готовое подключение

```
buro connect opencodez
```

connect устанавливает **skill и подключение начального контекста**. рядом со SKILL.md он кладёт system-prompt.md, готовый блок для ручного копирования. **системный промпт connect не редактирует.** для OpenCode/OpenCodez выбирается профиль из OPENCODE_CONFIG_DIR, если переменная задана; `--path <профиль>` имеет приоритет.

skill и блок системного промпта лежат вместе в skills/buro/. каждый объясняет основы и команды. можно использовать один из них или оба. в OpenCodez свой блок добавляется через библиотеку системных промптов или редактированием личного промпта.

другой агент

```
buro connect file --path /path/to/AGENTS.md
```

эта явная команда добавляет инструкции в указанный файл. автоматический стартовый контекст настраивается отдельно, если готового подключения нет.

что объясняют инструкции

как найти нужную сущность, прочитать запись и учитывать её правила. как посмотреть действующие типы и поля. как подготовить правку, проверить diff и сохранить её через push.

сведения проверяются при противоречиях или признаках устаревания. неизвестное остаётся неизвестным. обсуждение идеи не считается решением, а история сессии не сохраняется вместо полезных фактов. сохранённая запись не даёт разрешения на действия сверх запроса юзера.

контекст в начале сессии

при старте подключённой сессии она получает сведения о текущем устройстве и его записях через **buro current**. подключение запускает команду и добавляет вывод до первого ответа нейросети. в OpenCode/OpenCodez это происходит при первом запросе сессии, один раз за время её жизни в работающем процессе.

текущее устройство - то, где работает харнесс. передаются его запись, если она есть, и список объектов с расположением на нём. имя берётся из настройки current_context, иначе - из короткого системного имени устройства. отсутствие записи host нормально.

лимит списка задаётся через current_limit в config.json; по умолчанию - 100. если объектов больше, вывод сообщает об ограничении. это часть памяти, а не перечень всех её записей.

> **нужную сущность выбирает сама нейросеть по задаче и разговору.** BURO выдаёт запрошенные записи. отдельный определитель активного проекта для этого не нужен.

Харнесс	Начальный контекст
OpenCode / OpenCodez	Плагин передаёт current в системный контекст. После установки плагина нужен перезапуск.
локальный Codex	Разрешённый SessionStart hook передаёт current при startup или clear.
Claude Code	Командный SessionStart hook передаёт current при startup или clear.

продолжение и сжатие терминальной сессии не запускают эти hooks заново. при необходимости агент обновляет сведения командой `buro current`.

если BURO недоступен, чат всё равно начинается. агент получает сообщение об ошибке, а не «пустую память». подключения входят в пакет и обновляются через `buro update`. подключения Codex/Claude рассчитаны на местные харнессы; подробности совместимости - в руководстве по агентам.

команды и записи

справка объясняет программу и действующую модель памяти. содержимое конкретных записей открывается отдельно.

```
buro help
```

help показывает команды, порядок чтения и правки, подключение агента и перенос памяти. там же видны версия BURO и настроенная память: путь к местной базе или адрес сервера.

ниже - тип по умолчанию и все активные типы с пометками «коробочный» или «свой». для каждого раскрыты поля и вложенные поля: формат, обязательность и пояснения. это определения именно подключённой памяти.

Команда	Описание
`buro <id или имя>`	Открыть одну запись: правила, расположения и остальные выводимые поля.
`buro search <текст>`	Найти по ID, имени и псевдонимам. Содержимое остальных полей не обыскивается.
`buro list [тип]`	Показать ID, имена и типы записей.
`buro current`	Получить сведения о текущем устройстве и ограниченный список связанных объектов.

имена и страницы результатов

точный ID имеет приоритет. если имя неоднозначно, видны ID кандидатов. если ID совпал с названием команды, запись можно открыть через `buro get <id>`.

list и search принимают `--limit` и `--offset`. например:

```
buro list project --limit 100 --offset 100
```

при недоступной памяти help показывает справку и сообщает, что действующие определения получить не удалось. коробочные типы вместо них не подставляются. агент не загружает весь реестр, когда достаточно одного запроса.

обновление памяти

по умолчанию агент пишет в память в конце выполненной работы, когда есть что полезного сохранить, или по прямой просьбе юзера. другой момент записи можно задать в своих инструкциях.

каждая правка - отдельный файл

Команда	Описание
`buro draft new <id> [тип]`	Создать запись.
`buro draft pull <id>`	Изменить запись.
`buro draft delete <id>`	Подготовить удаление записи.

команда печатает путь к временному YAML-файлу. агент или юзер правит его, сохраняет служебные метаданные **__buro** и подставляет напечатанный путь вместо `<файл>`:

```
buro draft diff <файл>
buro draft push <файл>
```

diff показывает и проверяет изменение. push повторяет проверку значений и исходных версий, создаёт снимок, затем сохраняет правку. успешный push удаляет только свой файл. неудачный push оставляет его. `buro draft clear <файл>` намеренно отбрасывает выбранный черновик.

что происходит при одновременной работе

две сессии могут править разные записи. если кто-то уже изменил ту же запись или её определение типа, устаревший черновик отклоняется. нужно получить свежий, перенести свою правку и снова посмотреть diff. та же защита действует при правке самих типов.

сохраняется смысл, а не пересказ чата

нужны подтверждённые сведения, решения и правила, которые пригодятся дальше. учитывается весь разговор, включая последние уточнения и отменённые решения. дубли, временные планы и журнал сессии не нужны. если полезного изменения нет, писать нечего.

коробочные типы

при новой установке доступны три готовых типа. можно пользоваться подходящими или вести только записи собственных типов. заполнять все коробочные типы не требуется.

Тип	Описание
`project`	Самостоятельное дело или продукт: программа, сайт, бот, книга или исследование. Дополнительное поле repo - ссылка на репозиторий, если он есть.
`service`	Технический компонент рабочей среды: DNS, VPN, прокси, хранилище или MCP-инструмент. Дополнительное поле repo - репозиторий кода или конфигурации.
`host`	Компьютер, сервер или виртуальная машина. Кроме общих полей есть addresses, ssh_target и git_identities.

проект может иметь работающую версию и сведения об эксплуатации. у сервиса могут быть свой код и репозиторий. тип выбирается по роли объекта в работе юзера; папка и место установки его не определяют.

расположения и связи

locations - список расположений. у каждого могут быть host (устройство), path (путь), url (сетевой адрес) и purpose (что здесь находится). например, исходники сайта лежат на ноутбуке, рабочая установка - на сервере, а публичная страница открывается по URL.

related - справочные ссылки. id указывает на запись BURO, path - на файл, url - на веб-ресурс. note поясняет, зачем нужна ссылка.

подпись своего типа можно поменять отдельно

label - подпись типа. у своего типа её можно заменить, не меняя ID, поля и записи. изменение формата поля - другая операция: BURO сначала проверяет сохранённые значения.

коробочные определения не редактируются напрямую. для своей версии создают независимую копию через `buro types new my-project --from project`.

проект и сервис

проект - продукт или самостоятельное дело. сервис - технический компонент рабочей среды.

сервисы обычно относятся к локальной рабочей среде. подключённые внешние инструменты тоже подходят: например, Cloudflare как часть инфраструктуры. само расположение программы не определяет её тип.

Сведения	project	service
назначение	Назначение продукта, его пользователи и требования к результату.	Какую техническую возможность предоставляет компонент.
исходники	Код, материалы, сборка и устройство продукта.	Конфигурация, установочные скрипты и собственный код сервиса.
работающая версия	Где работает проект, как его публиковать, обновлять и восстанавливать.	Где работает компонент среды, как его настраивать и обслуживать.
важные правила	Требования к продукту, принятые решения и ограничения работы над ним.	Правила доступа, подключения, совместимости и эксплуатации.
связи	Какие инструменты и сервисы использует проект.	Кто пользуется сервисом и от каких компонентов он зависит.

у работающего бота код, конфигурация, данные и обновления могут оставаться в одной проектной записи. отдельная запись service нужна для самостоятельно обслуживаемого компонента со своей ответственностью.

например, запись проекта сайта содержит сведения о содержании, сборке и публикации. Caddy - программа, которая принимает веб-запросы и направляет их к нужному сайту или приложению. в его сервисной записи описаны правила маршрутизации, работы с сертификатами и доступа. записи связаны через related; общие поля у обоих типов одинаковые.

поля коробочных записей

у трёх готовых типов есть восемь общих полей. они необязательны и означают одно и то же во всех трёх типах:

Поле	Описание
`summary`	Текст. Что это за объект, зачем он существует и что делает.
`important`	Список. Проверенные факты, решения и требования к результату. Здесь же - условия согласования действий с юзером.
`avoid`	Список. Остальные запреты и ограничения: например, не менять формат файлов или не запускать тяжёлую сборку на этом устройстве.
`locations`	Список расположений: исходники, рабочая установка, данные, материалы или веб-страница.
`related`	Справочные ссылки: id записи, path к файлу или url ресурса. note объясняет связь.
`read_first`	Список основных документов и инструкций, с которых стоит начать работу.
`aliases`	Список других названий объекта, по которым его могут искать.
`notes`	Текст. Подробности, которым недостаточно короткого summary.

у project и service есть ещё необязательное текстовое поле **repo** - ссылка на репозиторий с кодом, материалами или конфигурацией объекта.

> **условие согласования всегда хранится в important.** «получить согласие перед публикацией» и «не публиковать без согласия» означают одно правило. формулировка с «не» не переносит его в avoid. дублировать правило в двух полях не нужно.

id - постоянный идентификатор записи, name - её имя, kind - тип. их наличие обязательно; версию записи ведёт само BURO. у своего типа могут быть другие поля и другие пояснения.

поля устройства

host - запись о компьютере, сервере или виртуальной машине. у неё есть все восемь общих полей из предыдущего раздела и ещё три. **все три необязательны.**

Поле	Описание
`addresses`	Список сетевых адресов компьютера. Например, адрес в домашней сети и адрес в VPN. Можно указать один адрес или несколько.
`ssh_target`	Текст. Имя или адрес для подключения по SSH - удалённой работы с компьютером через терминал. Например, home-server для команды ssh home-server.
`git_identities`	Список вариантов для работы с Git и GitHub: имя и email автора изменений, аккаунт GitHub и пояснение, когда его использовать.

например, один вариант git_identities нужен для личных проектов, другой - для рабочих. у элемента могут быть user_name, user_email, gh_cli_user и instructions. имя и email указывают автора изменений в истории Git. аккаунт GitHub - учётная запись для работы с репозиториями на GitHub. это разные сведения.

куда писать остальное

в summary можно написать «рабочий ноутбук». в notes - сведения об операционной системе и установленных программах. требования к работе идут в important, запреты и ограничения - в avoid. значимые папки компьютера можно указать в locations.

запись устройства необязательна

для current достаточно имени устройства. locations.host не требует существующей записи host. когда запись устройства есть, агент получает и её сведения; когда её нет, остаётся список связанных объектов.

имена и постоянные ID

id - уникальный идентификатор записи. это может быть понятное текстовое обозначение, например personal-site. числовой ID не требуется. name - отображаемое название, например «Сайт».

ID постоянен: обычной правкой его изменить нельзя. название можно переименовать, ссылки по ID продолжат вести на ту же запись. уникальность ID действует на всю память, независимо от типа записи.

Ситуация	Что происходит
ID уже занят	Вторая запись с таким ID не создаётся. Если речь о том же объекте, изменяется существующая запись. Другому объекту назначается другой ID.
названия совпадают	Это допустимо. Два разных сайта могут называться «Сайт», но иметь ID personal-site и company-site. При неоднозначном поиске BURO показывает кандидатов с их ID.
новый объект после удаления	Название может совпадать с названием удалённой записи, но ID нужен новый. Удалённый ID остаётся закреплённым за прежним объектом.
возвращается прежний объект	Восстановление из копии возвращает прежние данные и ID. Обычное создание новой записи с удалённым ID отклоняется.

> например, на старый сайт с ID my-site остались ссылки в других записях. если отдать этот ID новому сайту, ссылки начнут вести на другой объект. запрет повторной выдачи ID предотвращает такую подмену.

точный ID имеет приоритет перед совпадением с чужим названием или псевдонимом. названия и псевдонимы помогают найти запись; для ссылки используется её точный ID.

у типов своё правило: **один ID - одно действующее определение**. копия типа получает другой ID и остаётся независимой от исходного определения.

связи между записями

locations: где находится объект

```
locations:
  - host: laptop
    path: /home/user/projects/my-site
    purpose: Исходники
  - url: https://example.com
    purpose: Публичный сайт
```

новый path должен быть абсолютным. если у нового пути не указан host, BURO добавляет текущее устройство клиента; добавление видно в diff. в locations.host указывается имя устройства. создавать для него отдельную запись host необязательно.

related: справочные ссылки

```
related:
  - id: voice-service
    note: Публикует аудиоверсии статей.
  - url: https://example.com/docs
    note: Документация инструмента.
```

у элемента коробочного related выбирается id, path или url; note поясняет ссылку. удаление связанной записи не меняет остальные записи. при обращении к удалённой BURO сообщает, что она не найдена. исправить ссылку можно обычной правкой через diff и push.

ref: ссылка с проверкой

поле формата **ref** ссылается на существующую запись. например, у сайта поле «основной сервер» указывает на home-server. пока эта ссылка используется, BURO не даст удалить home-server. сначала нужно убрать ссылку или выбрать другую цель. формат ref доступен для собственных типов; использовать его необязательно.

создать свой тип

коробочные определения поставляет автор BURO. свои создаёт и меняет юзер или агент; BURO хранит и подключает их само. папку и пути к определениям выбирать не нужно.

например, нужен тип проектов только с одним полем important. для этого подходит копия готового project:

```
buro types new my-project --from project
```

команда печатает путь к временному YAML-файлу. в нём оставляют нужные поля и их пояснения, сохраняя __buro. затем:

```
buro draft diff <файл>
buro draft push <файл>
```

после push **my-project сразу доступен для записей**. менять config.json или отдельно запускать init не нужно. коробочный project остаётся прежним, его обновления не меняют копию.

остальные действия

```
buro types new <тип>
buro types edit <тип>
buro types delete <тип>
```

каждая команда готовит отдельный черновик. diff и push работают с его путём так же, как при правке записи. используемый тип можно удалить после переноса или явного удаления его записей; проверяемые ссылки тоже должны оставаться корректными.

> **копия - самостоятельное определение.** поля не подмешиваются из других типов и не наследуются скрыто. внутри одного типа имя поля уникально; в разных типах оно может иметь разные определения.

--from предпочитает поставляемый коробочный шаблон, если такой ID есть; иначе копирует активный свой тип. у своего типа label можно менять отдельно от полей.

как описать поля

у типа есть id, необязательная подпись label и объект fields. в fields перечисляются собственные поля типа. **тип записи** book и **формат значения** string-list - разные вещи.

Формат	Описание
`string / text`	Строка / развёрнутый текст. Например, название издательства или требования к стилю.
`boolean`	Да или нет: YAML-значения true и false.
`integer / number`	Безопасное целое число / конечное число с дробной частью.
`string-list`	Список непустых строк: правила, псевдонимы, названия документов.
`record / record-list`	Вложенные поля / список таких объектов. Их состав описывается через fields. Пустые элементы списка отклоняются.
`ref`	ID существующей записи. Необязательный target_kind ограничивает тип цели.

что поясняет само поле

type задаёт формат. **guide** объясняет агенту, что сюда писать. **required: true** делает поле обязательным. без этой пометки оно необязательно. вложенные поля описываются тем же способом.

default задаёт значение по умолчанию. если изменение типа добавляет такие значения в прежние записи, они сохраняются вместе с правкой. последующая смена default не переписывает уже сохранённые значения.

section группирует поля в выводе; их порядок берётся из определения. packet: false скрывает поле, включая вложенное, из обычного вывода записи, но не удаляет данные и не является защитой секрета.

поля с дополнительным поведением

aliases формата string-list участвует в поиске по псевдонимам. стандартный locations связывает запись с устройством. эти поля необязательны. остальные поля не приобретают поведения только из-за своего названия. неизвестные поля и неверные форматы отклоняются.

свой тип: книга

юзер ведёт несколько книг. для каждой нужно помнить требования к тексту и факт публикации. команда **buro types new book** выдаёт файл; в нём можно описать такой тип:

```
id: book
label: Книга
fields:
  editorial_rules:
    type: string-list
    guide: Принятые требования к тексту книги.
  published:
    type: boolean
    guide: Опубликована ли книга.
```

после diff и push тип готов. **buro draft new city-book book** создаёт черновик конкретной книги:

```
id: city-book
name: История города
kind: book
editorial_rules:
  - Сохранять разговорную речь в цитатах.
  - Согласовать финальный текст с автором.
published: false
```

это содержимое записи; служебные данные __buro созданного черновика сохраняются. его тоже проверяют через diff и применяют через push.

> юзер говорит: «продолжим править “Историю города”». агент находит city-book, читает требования и учитывает их в работе. новое принятое требование добавляет в эту же запись.

guide помогает выбрать правильное поле. формат boolean не даст записать «вроде бы» вместо true или false. но была ли книга действительно опубликована, BURO за агента не выясняет.

перенести запись в другой тип

тип записи можно сменить обычной правкой через draft pull, diff и push. **ID остаётся прежним.** меняются kind и состав содержательных полей.

пример: проект становится книгой

раньше книга была записью project. теперь создан собственный book. в старой записи есть important и repo; в новой нужны editorial_rules и published. совпадение смысла полей решают юзер или агент.

Шаг	Что сделать
открыть черновик	Получить текущую запись через draft pull. Проверить её значения и определение нового типа.
подготовить перенос	Сменить kind на book. Перенести правила из important в editorial_rules, если это действительно требования к книге.
решить судьбу остального	Для заполненного repo в новом типе места нет. Можно добавить подходящее поле в book, сохранить значение в другом согласованном месте или явно удалить его.
проверить и сохранить	Diff показывает смену типа, удаляемые и добавляемые поля. Push проверяет новую запись и ссылки, затем применяет правку целиком.

> **BURO не угадывает соответствие полей.** смена kind сама по себе не отбрасывает неподходящие значения. если они оставлены в неизвестных полях, push отклоняет правку. сознательное удаление значения видно в diff.

если на запись ссылается поле ref, ограниченное старым типом, перенос блокируется до исправления этой ссылки. справочные ссылки related по прежнему ID сохраняют смысл.

так же разбираются переименование поля и другие несовместимые изменения. массовый перенос сначала проверяется на копии; import не используется вместо обычной правки одной записи.

сохранность данных

> **несовместимое изменение не ломает рабочую память.** BURO отклоняет правку, объясняет причину и продолжает работать с прежними записями и определениями.

изменение типа проверяется заранее

новое определение проверяется по сохранённым записям. например, в поле записано «пять или шесть», а новый формат принимает одно число. BURO не выбирает 5 или 6. агент или юзер разбирается со значением и повторяет изменение. частично перенесённая память не становится рабочей.

импорт заменяет всю память

export сохраняет записи, определения типов и зарезервированные удалённые ID. import заменяет ими весь реестр. перед применением доступен просмотр: сколько записей есть сейчас, сколько придёт и какие исчезнут. это не команда добавления нескольких записей.

```
buro export memory.yaml
buro import memory.yaml --dry-run
buro import memory.yaml
```

входящий файл проверяется целиком. перед заменой создаётся копия. ошибка проверки или копирования отменяет операцию. замена записей и модели выполняется целиком; сбой возвращает прежнее состояние.

правильный по формату пустой импорт может очистить реестр. программа не знает, случайно ли выбран файл. ошибочную допустимую замену отменяют восстановлением из копии.

> перед изменением записи, типа или всей памяти создаётся снимок. если копию создать не удалось, правка не применяется. после import или восстановления прежние черновики больше не принимаются; их нужно подготовить заново.

каждая содержательная правка копирует всю базу. это расходует I/O и место при росте памяти; BURO подходит для устойчивых фактов, а не для потока событий или больших двоичных данных.

черновик не является резервной копией

это временный файл с отдельным путём. очистка временных каталогов может удалить старые черновики. нужные незавершённые правки сохраните отдельно. неудачный push оставляет свой файл для исправления.

копии и восстановление

для отмены недавней ошибки и возврата на несколько дней назад по умолчанию хранятся три независимые группы копий:

Копии	Что хранится
каждые 3 часа	Последние 8 копий, с интервалом не менее трёх часов.
раз в сутки	Последние 8 суточных копий.
перед изменениями	Последние 20 снимков перед правками записей, типов и всей памяти. Сюда же идут ручные копии.

обычные правки не вытесняют суточные копии. старая копия удаляется только после успешного создания новой в той же группе.

init на устройстве с местной базой настраивает штатный планировщик ОС: user systemd timer в Linux, LaunchAgent в macOS, задачу пользователя в Windows. клиенты общей памяти расписание не устанавливают. при недоступном планировщике init сообщает **NOT enabled** и даёт команду для ручного расписания. выключенное устройство копий не делает.

ручной снимок и восстановление

```
buro backup
buro restore /absolute/path/to/snapshot.sqlite3 --dry-run
buro restore /absolute/path/to/snapshot.sqlite3
```

перед восстановлением остановите API и других писателей. проверьте выбранный снимок через --dry-run, восстановите его, прочитайте `buro list` и известную запись, затем возобновите работу. старые черновики нужно создать заново.

снимок содержит записи, типы, удалённые ID, версию BURO и настройки контекста устройства с базой. восстанавливаются current_context и current_limit. пути хранения и настройки транспорта целевого устройства остаются его собственными.

> **если сломается диск.** копии на том же диске спасают от ошибочной правки, но погибнут вместе с диском. для защиты от его поломки сохраняйте копии на другом устройстве или в отдельном хранилище.

местная и общая память

если api_url в config.json не указан, бюро работает с местной SQLite. если указан - обращается к этому серверу по HTTP. **для общей памяти достаточно одной базы.** клиенты читают и меняют те же записи и типы.

на устройстве с базой, в доверенной частной сети:

```
buro serve --host 0.0.0.0 --port 8765
```

на остальных устройствах - в ~/.config/buro/config.json:

```
{
  "api_url": "http://home-server:8765",
  "current_context": "laptop",
  "current_limit": 100
}
```

по умолчанию serve слушает только 127.0.0.1. BURO_CONFIG выбирает файл настроек; BURO_ROOT - местный корень данных. переменные окружения важнее JSON; полный список находится в руководстве по эксплуатации.

если сервер недоступен, BURO сообщает об ошибке. он не переключается на местную базу и не создаёт другую память взамен общей. при тайм-ауте записи сервер мог успеть применить изменение; перед повтором прочитайте нужную запись.

доступ и секреты

у встроенного сервера нет аутентификации, разделения прав и TLS. допущенный к API клиент может читать и менять общую память. доступ дают доверенным клиентам через ограниченную сеть, VPN, защищённый туннель или HTTPS-прокси.

ключи, пароли и токены можно хранить по тем же правилам, что и остальные сведения. бюро не маскирует значения автоматически: агент видит выводимые поля. база, экспорты, копии и обычный HTTP не шифруются самим BURO. packet: false не ограничивает доступ.

обновления и подробности

`buro update` **устанавливает последний стабильный релиз GitHub на этом устройстве.** `buro update --check` только проверяет. команда проверяет пакет, применяет совместимые обновления коробочных определений к существующей местной памяти и обновляет подключения агентов. свои типы и записи сохраняются.

после обновления перезапустите подключённых агентов и работающий API BURO. для старого нестандартного профиля один раз повторите connect, чтобы запомнить путь. версии до 3.1 сначала обновляются вручную через npm. системный промпт поддерживается вручную; готовые инструкции лежат в skills/buro/.

что делать при ошибке

Ситуация	Следующий шаг
API недоступен	Проверить api_url и связь с устройства, где работает агент.
нет начального контекста	Проверить buro current в окружении харнесса, подключение и разрешение hook.
черновик устарел	Получить свежий, перенести нужную правку и снова проверить diff.
не удалось создать копию	Проверить место и права в каталоге копий, затем повторить сохранённый черновик.

подробные руководства

- README: установка и главная точка входа.
- Модель: поля, ссылки, определения и переход с v2.
- Агенты: подключения, профили и требования харнессов.
- Эксплуатация: настройки, расписания и восстановление.
- Разработка: архитектура, HTTP API и выпуск пакета.

при переходе с v2 сначала прочитайте порядок обновления модели и проверьте копию памяти. прежние определения сохраняются как свои под прежними ID; новые шаблоны их не подменяют.

бюро проверяет структуру и согласованность изменений. какие факты верны, что действительно решено и какие правила стоит запомнить, определяют пользователь и агент.